lvalue and rvalue

the terms **lvalue** and **rvalue**

are used to classify expressions based on whether they can appear on the left-hand side or right-hand side of an assignment statement.

lvalue

- An **lvalue** refers to an object that occupies some identifiable location in memory (i.e., it has an address).
- Lvalues can appear on the left-hand side of an assignment operator ($=$) because they represent a storage location that can be assigned a value.
- Examples of lvalues include variables, array elements, and dereferenced pointers.

rvalue

- An **rvalue** refers to a value that is not an lvalue. It is often a temporary or constant value that does not have a memory address.
- Rvalues can appear on the right-hand side of an assignment operator (=) because they represent a value rather than a storage location.
- Examples of rvalues include constants, literals, and the results of expressions.

Pointers and Arrays

Pointers and Arrays

- There's a strong relationship between pointers and arrays in C.
- Any operation achievable by array subscripting can be done with pointers.
- Pointer version generally faster but harder to understand.

Pointers and Arrays

```
int a[10];  /* defines an array of size 10 */
int *pa;    /* pointer to an integer */
pa = &a[0]; /* pa is set to the address of a[0] */
x = *pa;    /* copy a[0] into x */
y = *(pa+1) /* copy a[1] into y */
z = *(pa+i) /* copy a[i] into z */
```

Indexing and Pointer Arithmetic

- Are very close.
- Value of a variable or expression of type array is the address of element zero of the array.
- Expressions like `pa+1` point to the next object, and `pa+i` points to the i-th object beyond `pa`.

Equivalence of Array Name and Pointer

```
pa = &a[0];  /* pa is set to the address of a[0] */
pa = a;      /* the same */
*(a+i)       /* Equivalent to a[i] */
&a[i]        /* Equivalent to a+i */
pa[i]        /* Equivalent to *(pa+i) */
```

Difference Between Array Name and Pointer

- A pointer is a variable; `pa=a` and `pa++` are legal.
- An array name is not a variable; `a=pa` and `a++` are illegal.

Array Name as Function Parameter

Array name can be an argument to a function. The location of the initial element is passed. Within the function, this argument is a local variable.

```
/* strlen: return length of string s */
int strlen(char *s)
{
    int n;
    for (n = 0; *s != '\0', s++)
        n++;
    return n;
}
```

As formal parameters in a function definition, `char s[]` and `char *s` are equivalent. The latter makes it more explicit that the variable is a pointer. When an array name is passed, a function can use it as either an array or a pointer.

Passing Part of an Array to a Function

```
int a[10];
f(&a[2]); /* or f(a+2) */
f(int arr[]) { ... } /* or f(int *arr) { ... } */
```

It is also possible to index backwards in an array; p[-1], p[-2] , and so on. It is illegal to refer to objects that are not within the array bounds.

Address Arithmetic

Address Arithmetic

- If `p` is a pointer to some element of an array, then `p++` increments `p` to point to the next element, and `p+=i` increments it to point `i` elements beyond where it currently does.
- C integrates pointers, arrays, and address arithmetic seamlessly.
- Illustration through a rudimentary storage allocator follows.

Rudimentary Storage Allocator

```
#define ALLOCSIZE 10000 /* size of available space */
static char allocbuf[ALLOCSIZE]; /* storage for alloc */
static char *allocp = allocbuf; /* next free position */

char *alloc(int n)
{
  if (allocbuf + ALLOCSIZE - allocp >= n) {
    allocp += n;
    return allocp - n; /* old p */
  } else {
    return 0; /* not enough room */
  }
}
```

Rudimentary Storage Allocator

```
void afree(char *p)
{
  if (p >= allocbuf && p < allocbuf + ALLOCSIZE)
    allocp = p;
}
```

Address Arithmetic - Continued

- Pointers and integers are not interchangeable, except for zero.
- Pointer arithmetic and comparisons have certain rules and restrictions.
- Valid pointer operations: (All else are illegal.)
  - assignment of pointers of the same type
  - adding or subtracting a pointer and an integer
  - subtracting or comparing two pointers to members of the same array
  - assigning or comparing to zero
- Illegal operations include adding two pointers, multiplying, dividing, shifting, masking them, adding float or double to them, and assigning a pointer of one type to a pointer of another type without a cast.

Example: strlen Using Pointer Arithmetic

```
/* strlen: return length of string s */
int strlen(char *s)
{
  char *p = s;
  while (*p != '\0')
    p++;
  return p - s;
}
```

Character Pointers and Functions

Character Pointers and Functions

- A string constant is an array of characters, terminated with the null character '\0'.
- String constants are often used as arguments to functions.
- Access to a string constant is through a character pointer.

String Constants and Pointers

Important difference between `char amessage[]` and `char *pmessage`.

```
char amessage[] = "now is the time"; /* an array */
char *pmessage = "now is the time"; /* a pointer */
```

`char amessage[]` is an array, just big enough to hold the sequence of characters and '\0'. Individual characters within the array may be changed but amessage will always refer to the same storage.
`char *pmessage` is a pointer, initialized to point to a string constant; the pointer may subsequently be modified to point elsewhere, but the result is undefined if you try to modify the string contents.

String Copy Function - Array Subscript Version

```
/* strcpy: copy t to s; array subscript version */
void strcpy(char *s, char *t)
{
  int i = 0;
  while ((s[i] = t[i]) != '\0')
    i++;
}
```

String Copy Function - Pointer Version

```
/* strcpy: copy t to s; pointer version */
void strcpy(char *s, char *t)
{
  while ((*s = *t) != '\0') {
    s++;
    t++;
  }
}
```

String Copy Function - Improved Pointer Version

```
/* strcpy: copy t to s; pointer version 2 */
void strcpy(char *s, char *t)
{
  while ((*s++ = *t++) != '\0')
    ;
}
```

String Copy Function - Abbreviated Pointer Version

```
/* strcpy: copy t to s; pointer version 3 */
void strcpy(char *s, char *t)
{
  while (*s++ = *t++)
    ;
}
```

String Comparison Function

- `strcmp(s, t)` compares the character strings `s` and `t`.
- Returns negative, zero, or positive if `s` is lexicographically less than, equal to, or greater than `t`.

String Comparison Function - Array Subscript Version

```
/* strcmp: return <0 if s<t, 0 if s==t, >0 if s>t */
int strcmp(char *s, char *t)
{
  int i;
  for (i = 0; s[i] == t[i]; i++)
    if (s[i] == '\0')
      return 0;
  return s[i] - t[i];
}
```

String Comparison Function - Pointer Version

```
/* strcmp: return <0 if s<t, 0 if s==t, >0 if s>t */
int strcmp(char *s, char *t)
{
  for ( ; *s == *t; s++, t++)
    if (*s == '\0')
      return 0;
  return *s - *t;
}
```

Push and Pop

- `*p++ = val; /* push val onto stack */`
- `val = *--p; /* pop top of stack into val */`